

➤ Technical Report: Contact Management System

1. Executive Summary

The **Contact Management System** is a high-utility console application that allows users to create, store, and manage a digital address book. Its primary strengths lie in its **robust data validation** via Regular Expressions (regex) and its use of **binary serialization** for efficient long-term data storage.

2. Architecture & Design Patterns

The system is built using two primary classes that separate data representation from logic:

2.1 The Contact Class (Data Model)

This class serves as the blueprint for a single entry. It encapsulates the core attributes:

- **Attributes:** Name, Phone, Email, and Physical Address.
- **Responsibilities:** Input gathering, individual display formatting, and serialization (writing/reading raw bytes to a file).

2.2 The ContactManager Class (Controller)

This class acts as the system's engine, managing a dynamic collection of contacts using the `std::vector` container.

- **Functionality:** Implements the CRUD (Create, Read, Update, Delete) cycle.
- **Search & Sort:** Utilizes C++ Standard Template Library (STL) algorithms like `std::sort` and lambda expressions for flexible data organization.

3. Key Technical Features

3.1 Advanced Data Validation

To ensure data integrity, the system employs **Regular Expressions (Regex)**. This prevents common errors like:

- **Phone Validation:** Ensures exactly 10 digits are entered.
- **Email Validation:** Validates the presence of an "@" symbol and a proper domain suffix (e.g., .com).
- **Buffer Management:** Uses `numeric_limits` to clear the input stream, preventing "skipping" bugs common in C++ console apps.

3.2 Binary File Persistence

Unlike standard text files, this system saves data in **binary format** (.dat).

- **Efficiency:** Binary files are smaller and faster to process by the machine.
- **Method:** It saves the length of the string followed by the actual characters, ensuring that even strings with spaces are reloaded perfectly.

3.3 Dynamic Search and Sort

The system provides high user-utility through:

- **Partial Search:** Users can find a contact by searching for a partial name or a phone number.
- **Multi-Criteria Sorting:** Allows the user to toggle between alphabetical (Name) or numerical (Phone) organization.

4. Operational Workflow

Feature	Logic Description
Add	Validates 4 fields of data; pushes to std::vector.
Search	Iterates through the vector using string::npos for partial matches.
Update	Locates an index by name and overwrites the object attributes.
Delete	Uses vector::erase with an iterator to remove data.
Storage	Automatically loads on startup and saves on exit.

```
#include <iostream>
#include <vector>
#include <fstream>
#include <algorithm>
#include <regex>
#include <iomanip>
#include <limits> // For input validation

using namespace std;

// ----- Contact Class -----
class Contact {
public:
    string name;
    string phone;
    string email;
    string address;

    void input() {
        cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Clear input buffer

        // Name input
        while (true) {
            cout << "Enter Name: ";
            getline(cin, name);
            if (!name.empty()) break;
            cout << "Name cannot be empty!\n";
        }
    }
};
```

```

// Phone input with validation
while (true) {
    cout << "Enter Phone (10 digits): ";
    getline(cin, phone);
    if (regex_match(phone, regex("\\d{10}"))) break;
    cout << "Invalid phone number! Must be 10 digits.\n";
}

// Email input with validation
while (true) {
    cout << "Enter Email: ";
    getline(cin, email);
    if (regex_match(email, regex(R"((\w+) (\.?)(\w*)@(\w+)\.(\w+))"))) break;
    cout << "Invalid email format!\n";
}

// Address input
while (true) {
    cout << "Enter Address: ";
    getline(cin, address);
    if (!address.empty()) break;
    cout << "Address cannot be empty!\n";
}
}

void display() const {
    cout << left << setw(20) << name
        << setw(15) << phone
        << setw(25) << email
        << setw(30) << address << "\n";
}

```

```

void save(ofstream &ofs) const {
    size_t len = name.size();
    ofs.write((char*)&len, sizeof(len));
    ofs.write(name.c_str(), len);

    len = phone.size();
    ofs.write((char*)&len, sizeof(len));
    ofs.write(phone.c_str(), len);

    len = email.size();
    ofs.write((char*)&len, sizeof(len));
    ofs.write(email.c_str(), len);

    len = address.size();
    ofs.write((char*)&len, sizeof(len));
    ofs.write(address.c_str(), len);
}

void load(ifstream &ifstream) {
    size_t len;
    char buffer[256];

    ifs.read((char*)&len, sizeof(len));
    ifs.read(buffer, len); buffer[len] = '\0'; name = buffer;

    ifs.read((char*)&len, sizeof(len));
    ifs.read(buffer, len); buffer[len] = '\0'; phone = buffer;

    ifs.read((char*)&len, sizeof(len));
    ifs.read(buffer, len); buffer[len] = '\0'; email = buffer;
}

```

```

        ifs.read((char*)&len, sizeof(len));
        ifs.read(buffer, len); buffer[len] = '\0'; address = buffer;
    }
};

// ----- Contact Manager -----
class ContactManager {
    vector<Contact> contacts;

    // Helper function to find contact index by exact name
    int findContactIndex(const string &key) const {
        for (size_t i = 0; i < contacts.size(); ++i) {
            if (contacts[i].name == key) return i;
        }
        return -1;
    }

public:
    void addContact() {
        Contact c;
        c.input();
        contacts.push_back(c);
        cout << "Contact added successfully!\n";
    }

    void displayContacts() const {
        if (contacts.empty()) {
            cout << "No contacts available.\n";
            return;
        }
    }
};

```

```

        cout << left << setw(20) << "Name"
             << setw(15) << "Phone"
             << setw(25) << "Email"
             << setw(30) << "Address" << "\n";
        cout << string(90, '-') << "\n";
        for (const auto &c : contacts) c.display();
    }

    void searchContact() const {
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        string key;
        cout << "Enter name or phone to search: ";
        getline(cin, key);

        if (key.empty()) {
            cout << "Search key cannot be empty!\n";
            return;
        }

        bool found = false;
        for (const auto &c : contacts) {
            if (c.name.find(key) != string::npos || c.phone == key) {
                c.display();
                found = true;
            }
        }

        if (!found) cout << "No contact found.\n";
    }
};

```

```

void updateContact() {
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    string key;
    cout << "Enter name of contact to update: ";
    getline(cin, key);

    int index = findContactIndex(key);
    if (index == -1) {
        cout << "Contact not found!\n";
        return;
    }

    cout << "Enter new details:\n";
    contacts[index].input();
    cout << "Contact updated successfully!\n";
}

void deleteContact() {
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    string key;
    cout << "Enter name of contact to delete: ";
    getline(cin, key);

    int index = findContactIndex(key);
    if (index == -1) {
        cout << "Contact not found!\n";
        return;
    }

    contacts.erase(contacts.begin() + index);
}

```

```

    cout << "Contact deleted successfully!\n";
}

void sortContacts() {
    int choice;
    cout << "Sort by: 1. Name 2. Phone\nChoice: ";
    while (!(cin >> choice) || (choice != 1 && choice != 2)) {
        cout << "Invalid choice! Enter 1 or 2: ";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    if (choice == 1)
        sort(contacts.begin(), contacts.end(), [](const Contact &a, const Contact &b){ return a.name < b.name; });
    else
        sort(contacts.begin(), contacts.end(), [](const Contact &a, const Contact &b){ return a.phone < b.phone; });

    cout << "Contacts sorted successfully!\n";
}

void saveToFile() const {
    ofstream ofs("contacts.dat", ios::binary);
    if (!ofs) {
        cout << "Error saving file!\n";
        return;
    }
    size_t size = contacts.size();
    ofs.write((char*)&size, sizeof(size));
    for (const auto &c : contacts) c.save(ofs);
}

```

```

void loadFromFile() {
    ifstream ifs("contacts.dat", ios::binary);
    if (!ifs) return;

    size_t size;
    ifs.read((char*)&size, sizeof(size));
    contacts.resize(size);
    for (auto &c : contacts) c.load(ifs);
}

};

// ----- Main -----
int main() {
    ContactManager cm;
    cm.loadFromFile();

    int choice;
    do {
        cout << "\n=== CONTACT MANAGEMENT SYSTEM ===\n";
        cout << "1. Add Contact\n2. Display Contacts\n3. Search Contact\n";
        cout << "4. Update Contact\n5. Delete Contact\n6. Sort Contacts\n7. Exit\n";
        cout << "Enter your choice: ";

        while (!(cin >> choice) || choice < 1 || choice > 7) {
            cout << "Invalid input! Enter number between 1-7: ";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }
    }

```

```

        switch (choice) {
            case 1: cm.addContact(); break;
            case 2: cm.displayContacts(); break;
            case 3: cm.searchContact(); break;
            case 4: cm.updateContact(); break;
            case 5: cm.deleteContact(); break;
            case 6: cm.sortContacts(); break;
            case 7: cm.saveToFile(); cout << "Contacts saved. Exiting...\n"; break;
        }
    } while (choice != 7);

    return 0;
}

```

5. Security and Stability Assessment

- **Stability:** The inclusion of `cin.clear()` and `cin.ignore()` makes the program highly resistant to "Input Mismatch" crashes (e.g., entering a letter when a number is expected).
- **Validation:** Regex-based email and phone checks provide a professional layer of data sanitization rarely seen in basic console applications.

6. Recommendations for Future Development

To transition this from a console utility to a production-grade tool, the following could be added:

1. **Encryption:** Adding a simple XOR or AES encryption layer to the `contacts.dat` file for privacy.
2. **Duplicate Detection:** Logic to prevent two contacts from having the exact same phone number.
3. **Export to CSV:** A feature to export the contact list into an Excel-compatible format.